

Competitive Programming

Section 66 Editorial: Master+ Mathematics Mixed Contest

Competitive Programming Course

2026-08-20

Table of contents

1	Coverage and progression	4
2	A. Primitive Necklaces	5
2.1	Derivation	5
2.2	Algorithm	5
2.3	Correctness proof	5
2.4	Complexity and tests	6
2.5	Full C++ solution	6
2.6	Full Python solution	7
3	B. Totient Interval	9
3.1	Reduction to the learned primitive	9
3.2	Correctness proof	9
3.3	Complexity and tests	9
3.4	Full C++ solution	9
3.5	Full Python solution	10
4	C. Wildcard Rotation	12
4.1	Count conflicts, not compatibility cases	12
4.2	Correctness proof	12
4.3	Complexity and tests	12
4.4	Full C++ solution	13
4.5	Full Python solution	14
5	D. Weighted Compositions	15
5.1	Sparse factors	15
5.2	Correctness proof	15
5.3	Complexity and tests	15
5.4	Full C++ solution	15
5.5	Full Python solution	16
6	E. Black-Box Walks	18
6.1	Why a scalar recurrence exists	18
6.2	Correctness proof	18
6.3	Complexity and tests	18
6.4	Full C++ solution	19
6.5	Full Python solution	21
7	F. GCD-One Size Spectrum	24
7.1	First transform: exact gcd by Mobius	24
7.2	Group equal counts and expose a convolution	24
7.3	Correctness proof	25
7.4	Complexity and tests	25
7.5	Full C++ solution	25
7.6	Full Python solution	26

8	Stress-testing strategy	29
9	What to carry forward	30

1 Coverage and progression

Problem	First model	Decisive compression
A	words grouped by minimal period	Mobius inversion, then rotation orbit size
B	a range as two prefixes	memoized harmonic quotient blocks
C	concrete conflicts at every rotation	five cyclic correlations
D	independent weighted choices	sparse generating-function product
E	one matrix-power entry	learn a scalar recurrence from $2n+1$ terms
F	fixed-size subsets divisible by d	Mobius sum becomes one factorial convolution

2 A. Primitive Necklaces

2.1 Derivation

Temporarily distinguish a starting position, so we count linear words. Let `primitive[d]` be the number of length-`d` words whose minimal period is `d`. Every length-`n` word has one minimal period `d` dividing `n`, and is obtained by repeating its primitive block. Therefore

$$c^n = \sum(\text{primitive}[d] \text{ for } d \text{ dividing } n).$$

Mobius inversion gives

$$\text{primitive}[n] = \sum(\mu[d] * c^{(n/d)} \text{ for } d \text{ dividing } n).$$

Every primitive length-`n` word has exactly `n` distinct rotations: equality after a smaller rotation would be a smaller period. Thus each primitive necklace contains exactly `n` linear representatives, and the answer is `primitive[n]/n`.

For binary length three, there are eight words. The two constant words have period one; the other six are primitive and form two orbits of three.

2.2 Algorithm

Sieve `mu` through the largest queried `n`. Enumerate each query's divisors in $O(\sqrt{n})$, evaluate the Mobius sum by modular exponentiation, and multiply by the modular inverse of `n`.

2.3 Correctness proof

Lemma 1. The Mobius sum counts exactly the linear words of minimal period `n`.

Proof. The cumulative divisor equation above partitions all words by their unique minimal period. Mobius inversion is the inverse of this divisor sum. QED.

Lemma 2. Each primitive rotation orbit has size `n`.

Proof. If two of its `n` rotations were equal, their nonzero displacement would be a period smaller than `n`, contradicting primitiveness. QED.

Dividing Lemma 1's labeled count by the orbit size from Lemma 2 counts every primitive necklace once.

2.4 Complexity and tests

The sieve costs $O(N)$ with the linear implementation. A query costs $O(\sqrt{n} \log n)$ for divisor enumeration and powers; memory is $O(N)$. Test one color, $n=1$, prime n , and brute rotation classes for tiny alphabets.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 1'000'000'007;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q)) return 0;
    vector<pair<int, int64>> queries(q);
    int maximum = 1;
    for (auto &[n, colors] : queries) {
        cin >> n >> colors;
        maximum = max(maximum, n);
    }

    vector<int> mu(maximum + 1), primes;
    vector<bool> composite(maximum + 1);
    mu[1] = 1;
    for (int value = 2; value <= maximum; ++value) {
        if (!composite[value]) primes.push_back(value), mu[value] = -1;
        for (int prime : primes) {
            if (1LL * value * prime > maximum) break;
            composite[value * prime] = true;
            if (value % prime == 0) {
                mu[value * prime] = 0;
                break;
            }
            mu[value * prime] = -mu[value];
        }
    }
}
```

```

    }
}

for (auto [n, colors] : queries) {
    int64 primitive_words = 0;
    for (int divisor = 1; 1LL * divisor * divisor <= n; ++divisor) {
        if (n % divisor) continue;
        primitive_words += mu[divisor] * power(colors, n / divisor);
        int other = n / divisor;
        if (other != divisor)
            primitive_words += mu[other] * power(colors, n / other);
        primitive_words %= MOD;
    }
    if (primitive_words < 0) primitive_words += MOD;
    cout << primitive_words * power(n, MOD - 2) % MOD << '\n';
}
}

```

2.6 Full Python solution

```

import math
import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    queries = [tuple(data[i : i + 2]) for i in range(1, 2 * data[0] + 1, 2)]
    maximum = max(n for n, _ in queries)
    mu = [0] * (maximum + 1)
    composite = bytearray(maximum + 1)
    primes: list[int] = []
    mu[1] = 1
    for value in range(2, maximum + 1):
        if not composite[value]:
            primes.append(value)
            mu[value] = -1
        for prime in primes:
            product = value * prime
            if product > maximum:
                break
            composite[product] = 1
            if value % prime == 0:
                mu[product] = 0
                break

```

```

        mu[product] = -mu[value]

output = []
for n, colors in queries:
    primitive_words = 0
    for divisor in range(1, math.isqrt(n) + 1):
        if n % divisor:
            continue
        primitive_words += mu[divisor] * pow(colors, n // divisor, MOD)
        other = n // divisor
        if other != divisor:
            primitive_words += mu[other] * pow(colors, n // other, MOD)
    answer = primitive_words % MOD * pow(n, MOD - 2, MOD) % MOD
    output.append(str(answer))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

3 B. Totient Interval

3.1 Reduction to the learned primitive

Let $\text{Phi}(x) = \sum_{i=1..x} \phi(i)$, with $\text{Phi}(0)=0$. The range answer is

```
Phi(R) - Phi(L-1).
```

The main work is the Section 64 summatory recurrence:

```
Phi(n) = n(n+1)/2
        - sum from i=2 to n of Phi(floor(n/i)).
```

If $q = \text{floor}(n/l)$, the quotient stays constant through $r = \text{floor}(n/q)$, so the whole interval contributes $(r-l+1) * \text{Phi}(q)$. Precompute the first million prefixes and memoize larger recursive arguments shared by all range endpoints.

For $[6, 10]$, the answer is $\text{Phi}(10) - \text{Phi}(5) = 32 - 10 = 22$.

3.2 Correctness proof

The identity $\sum_{d|m} \phi(d) = m$, summed over $m \leq n$, yields the summatory recurrence as proved in Section 64. Harmonic blocks partition every index $2..n$ without changing its quotient, so the grouped evaluation is equal to the recurrence. Finally, subtracting prefixes removes exactly terms below L , leaving $\phi(L) .. \phi(R)$.

3.3 Complexity and tests

The base sieve is $O(B \log \log B)$ time and $O(B)$ memory. Memoized large states use harmonic-block traversal rather than a linear scan. Test singleton ranges, $L=1$, endpoints around the precompute boundary, and random small ranges against direct gcd counting.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int LIMIT = 1'000'000;
constexpr int64 MOD = 1'000'000'007;
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    vector<int64> prefix(LIMIT + 1);
    iota(prefix.begin(), prefix.end(), 0);
    for (int prime = 2; prime <= LIMIT; ++prime)
        if (prefix[prime] == prime)
            for (int multiple = prime; multiple <= LIMIT; multiple += prime)
                prefix[multiple] -= prefix[multiple] / prime;
    for (int value = 1; value <= LIMIT; ++value)
        prefix[value] = (prefix[value] + prefix[value - 1]) % MOD;

    unordered_map<int64, int64> memo;
    function<int64(int64)> sum_phi = [&](int64 n) -> int64 {
        if (n <= 0) return 0;
        if (n <= LIMIT) return prefix[n];
        if (auto it = memo.find(n); it != memo.end()) return it->second;
        int64 answer = n % MOD * ((n + 1) % MOD) % MOD * ((MOD + 1) / 2) % MOD;
        for (int64 left = 2, right; left <= n; left = right + 1) {
            int64 quotient = n / left;
            right = n / quotient;
            answer = (answer - (right - left + 1) % MOD * sum_phi(quotient)) % MOD;
        }
        if (answer < 0) answer += MOD;
        return memo[n] = answer;
    };

    int q;
    cin >> q;
    while (q--) {
        int64 left, right;
        cin >> left >> right;
        cout << (sum_phi(right) - sum_phi(left - 1) + MOD) % MOD << '\n';
    }
}

```

3.5 Full Python solution

```

import sys

MOD = 1_000_000_007
LIMIT = 1_000_000

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

```

```

prefix = list(range(LIMIT + 1))
for prime in range(2, LIMIT + 1):
    if prefix[prime] == prime:
        for multiple in range(prime, LIMIT + 1, prime):
            prefix[multiple] -= prefix[multiple] // prime
for value in range(1, LIMIT + 1):
    prefix[value] = (prefix[value] + prefix[value - 1]) % MOD
memo: dict[int, int] = {}

def sum_phi(n: int) -> int:
    if n <= 0:
        return 0
    if n <= LIMIT:
        return prefix[n]
    if n in memo:
        return memo[n]
    answer = n * (n + 1) // 2 % MOD
    left = 2
    while left <= n:
        quotient = n // left
        right = n // quotient
        answer = (answer - (right - left + 1) * sum_phi(quotient)) % MOD
        left = right + 1
    memo[n] = answer
    return answer

output = []
index = 1
for _ in range(data[0]):
    left, right = data[index : index + 2]
    index += 2
    output.append(str((sum_phi(right) - sum_phi(left - 1)) % MOD))
print("\n".join(output))

if __name__ == "__main__":
    main()

```

4 C. Wildcard Rotation

4.1 Count conflicts, not compatibility cases

A position is invalid only when both characters are concrete and unequal. For each shift, compute:

```
both_concrete = number of positions where neither side is '?'
same_concrete = sum over A,C,G,T of equal-letter positions
conflicts = both_concrete - same_concrete
```

The shift is valid exactly when `conflicts=0`.

Each count is a cyclic correlation. For binary indicator arrays `left` and `right`, reverse `left`, duplicate `right`, convolve, and read coefficient `n-1+shift`. One correlation counts concrete/concrete pairs and four count equal concrete letters.

In the sample, shift two aligns `t` as `ACGT`; the wildcard in `s` accepts `C`, and the other three letters match. Every other shift has a concrete mismatch.

4.2 Correctness proof

Lemma 1. The correlation coefficient read for a shift counts exactly the positions where both chosen indicator predicates hold.

Proof. Reversed index `n-1-i` pairs at coefficient `n-1+r` with doubled index `i+r`, representing `t[(i+r) mod n]`. QED.

Lemma 2. A shift is compatible exactly when `both_concrete=same_concrete`.

Proof. Every same-concrete position is included among both-concrete positions. Their difference is precisely the positions with two distinct concrete letters, the only forbidden case. QED.

The scan therefore counts all and only valid shifts and records the smallest.

4.3 Complexity and tests

Five convolutions cost $O(n \log n)$ time and $O(n)$ memory. Test all wildcards, no valid shift, length one, and ties. A frequent bug is reading `n-1-r` instead of deriving `n-1+r`.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
#include "../65_polynomial_algorithms_recurrences/ntt.hpp"
using namespace std;
using course_ntt::convolution;

vector<int> correlation(const vector<int> &left, const vector<int> &right) {
    int n = left.size();
    vector<int> reversed(n), doubled(2 * n);
    for (int i = 0; i < n; ++i) {
        reversed[n - 1 - i] = left[i];
        doubled[i] = doubled[i + n] = right[i];
    }
    vector<int> product = convolution(reversed, doubled);
    vector<int> result(n);
    for (int shift = 0; shift < n; ++shift)
        result[shift] = product[n - 1 + shift];
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    string s, t;
    if (!(cin >> n >> s >> t)) return 0;
    vector<int> concrete_s(n), concrete_t(n);
    for (int i = 0; i < n; ++i) {
        concrete_s[i] = s[i] != '?';
        concrete_t[i] = t[i] != '?';
    }
    vector<int> both = correlation(concrete_s, concrete_t), matches(n);
    for (char letter : string("ACGT")) {
        vector<int> in_s(n), in_t(n);
        for (int i = 0; i < n; ++i) {
            in_s[i] = s[i] == letter;
            in_t[i] = t[i] == letter;
        }
        vector<int> same = correlation(in_s, in_t);
        for (int shift = 0; shift < n; ++shift) matches[shift] += same[shift];
    }
    int count = 0, first = -1;
    for (int shift = 0; shift < n; ++shift)
        if (both[shift] == matches[shift]) {
            if (first == -1) first = shift;
            ++count;
        }
    cout << count << ' ' << first << '\n';
}
```

4.5 Full Python solution

```
import sys
from pathlib import Path

SECTION_65 = Path(__file__).resolve().parents[3] / "65_polynomial_algorithms_recurrences"
sys.path.insert(0, str(SECTION_65))
from ntt import convolution

def correlation(left: list[int], right: list[int]) -> list[int]:
    n = len(left)
    product = convolution(left[::-1], right + right)
    return product[n - 1 : 2 * n - 1]

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n = int(data[0])
    s, t = data[1].decode(), data[2].decode()
    both = correlation([c != "?" for c in s], [c != "?" for c in t])
    matches = [0] * n
    for letter in "ACGT":
        same = correlation([c == letter for c in s], [c == letter for c in t])
        for shift in range(n):
            matches[shift] += same[shift]
    valid = [shift for shift in range(n) if both[shift] == matches[shift]]
    print(len(valid), valid[0] if valid else -1)

if __name__ == "__main__":
    main()
```

5 D. Weighted Compositions

5.1 Sparse factors

Type i contributes one of $0, w_i, 2w_i, \dots, a_i w_i$, so encode it by

$$P_i(x) = 1 + x^{w_i} + x^{2w_i} + \dots + x^{a_i w_i}.$$

The coefficient of x^K in the product chooses exactly one legal exponent per type and counts choices whose weights sum to K . Terms above K can be discarded because all future exponents are nonnegative.

Use a min-heap of current polynomial lengths and repeatedly convolve the two shortest, truncating to $K+1$ coefficients. This avoids repeatedly transforming a nearly complete product with a tiny factor.

For the sample, the solutions are $(x_1, x_2, x_3) = (2, 1, 1), (0, 2, 1), (1, 0, 2)$.

5.2 Correctness proof

Induct over multiplied types. The coefficient at degree s counts partial choices totaling s ; convolution with the next sparse factor appends each legal count exactly once. Associativity makes the heap order irrelevant. Discarded degrees exceed K and can never decrease, so they cannot influence the requested coefficient. Thus the final degree- K coefficient is exact.

5.3 Complexity and tests

With balanced merges and truncation, the course bound is $O(K \log K \log n)$ transform work, plus construction of the stated total number of nonzero factor coefficients. Memory is $O(K+n)$ live coefficients. Test $K=0$, weights larger than K , zero caps, and compare with weighted knapsack DP.

5.4 Full C++ solution

```
#include <bits/stdc++.h>
#include "../65_polynomial_algorithms_recurrences/ntt.hpp"
using namespace std;
using course_ntt::convolution;

int main() {
```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);
int n, target;
if (!(cin >> n >> target)) return 0;
struct Compare {
    bool operator()(const vector<int> &a, const vector<int> &b) const {
        return a.size() > b.size();
    }
};
priority_queue<vector<int>, vector<vector<int>>, Compare> heap;
for (int i = 0, weight, cap; i < n; ++i) {
    cin >> weight >> cap;
    int maximum = static_cast<int>(min(1LL * target, 1LL * weight * cap));
    vector<int> polynomial(maximum + 1);
    for (int value = 0; value <= maximum; value += weight) polynomial[value] = 1;
    if (polynomial.size() > 1) heap.push(move(polynomial));
}
if (heap.empty()) {
    cout << (target == 0) << '\n';
    return 0;
}
while (heap.size() > 1) {
    auto a = heap.top(); heap.pop();
    auto b = heap.top(); heap.pop();
    heap.push(convolution(a, b, target + 1));
}
const auto &answer = heap.top();
cout << (target < static_cast<int>(answer.size())) ? answer[target] : 0 << '\n';
}

```

5.5 Full Python solution

```

import heapq
import sys
from pathlib import Path

SECTION_65 = Path(__file__).resolve().parents[3] / "65_polynomial_algorithms_recurrences"
sys.path.insert(0, str(SECTION_65))
from ntt import convolution

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, target = data[:2]
    heap: list[tuple[int, int, list[int]]] = []
    index = 2

```

```

serial = 0
for _ in range(n):
    weight, cap = data[index : index + 2]
    index += 2
    maximum = min(target, weight * cap)
    polynomial = [0] * (maximum + 1)
    for value in range(0, maximum + 1, weight):
        polynomial[value] = 1
    if len(polynomial) > 1:
        heapq.heappush(heap, (len(polynomial), serial, polynomial))
        serial += 1
if not heap:
    print(int(target == 0))
    return
while len(heap) > 1:
    _, _, left = heapq.heappop(heap)
    _, _, right = heapq.heappop(heap)
    product = convolution(left, right, target + 1)
    heapq.heappush(heap, (len(product), serial, product))
    serial += 1
answer = heap[0][2]
print(answer[target] if target < len(answer) else 0)

if __name__ == "__main__":
    main()

```

6 E. Black-Box Walks

6.1 Why a scalar recurrence exists

Let A be the weighted adjacency matrix and let v_k contain total walk weights from s to every vertex after k edges. Then

```
v_(k+1) = A * v_k
answer sequence z_k = component t of v_k.
```

Matrix exponentiation would cost roughly $O(n^3 \log K)$. Cayley–Hamilton says the n by n matrix satisfies its degree- n characteristic polynomial. Multiplying that matrix identity by the starting vector and selecting the target component proves that z_k satisfies a linear recurrence of order at most n .

We do not compute the characteristic polynomial. Generate $2n+1$ scalar terms by sparse edge DP, recover their minimal recurrence with Berlekamp–Massey, and evaluate term K by polynomial reduction.

In the sample, a length-five walk moves from 1 to 2 on exactly one of its five edges; loops fill the other positions, giving five walks.

6.2 Correctness proof

Lemma 1. The generated term z_k is the sum of weights of all length- k walks from s to t .

Proof. Initially only the empty walk at s has weight one. Every transition extends every existing walk by one outgoing edge and multiplies its weight; summing at destinations counts all extensions exactly once. QED.

Lemma 2. The infinite scalar sequence has recurrence order at most n .

Proof. Apply the Cayley–Hamilton matrix relation to every $A^k v_0$, then take target component. This yields the same degree-at-most- n relation among consecutive scalar terms. QED.

Berlekamp–Massey recovers a minimal order- r recurrence from at least $2r$ correct terms. We generate more than $2n$, hence enough by Lemma 2. Section 65’s polynomial evaluator then returns the exact K -th recurrence term, which is the desired walk sum by Lemma 1.

6.3 Complexity and tests

Generating terms costs $O(nm)$. Recovery costs $O(n^2)$ and evaluation $O(n^2 \log K)$; memory is $O(n+m)$. Test $K=0$, no edges, parallel edges, self-loops, and compare small K with direct walk DP.

6.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
constexpr int64 MOD = 998244353;

int64 power(int64 base, int64 exponent) {
    int64 result = 1;
    while (exponent) {
        if (exponent & 1) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1;
    }
    return result;
}

vector<int64> berlekamp_massey(const vector<int64> &sequence) {
    vector<int64> current{1}, saved{1};
    int length = 0, shift = 1;
    int64 saved_discrepancy = 1;
    for (int position = 0; position < static_cast<int>(sequence.size()); ++position) {
        int64 discrepancy = sequence[position];
        for (int i = 1; i <= length; ++i)
            discrepancy = (discrepancy + current[i] * sequence[position - i]) % MOD;
        if (discrepancy == 0) {
            ++shift;
            continue;
        }
        vector<int64> previous = current;
        int64 scale = discrepancy * power(saved_discrepancy, MOD - 2) % MOD;
        if (current.size() < saved.size() + shift)
            current.resize(saved.size() + shift);
        for (int i = 0; i < static_cast<int>(saved.size()); ++i) {
            current[i + shift] = (current[i + shift] - scale * saved[i]) % MOD;
            if (current[i + shift] < 0) current[i + shift] += MOD;
        }
        if (2 * length <= position) {
            length = position + 1 - length;
            saved = move(previous);
            saved_discrepancy = discrepancy;
            shift = 1;
        } else {
            ++shift;
        }
    }
    vector<int64> recurrence(length);
    for (int i = 0; i < length; ++i)
        recurrence[i] = (MOD - current[i + 1]) % MOD;
}
```

```

    return recurrence;
}

vector<int64> combine(const vector<int64> &a, const vector<int64> &b,
                    const vector<int64> &recurrence) {
    int order = recurrence.size();
    vector<int64> product(2 * order - 1);
    for (int i = 0; i < order; ++i)
        for (int j = 0; j < order; ++j)
            product[i + j] = (product[i + j] + a[i] * b[j]) % MOD;
    for (int degree = 2 * order - 2; degree >= order; --degree)
        for (int back = 1; back <= order; ++back)
            product[degree - back] = (
                product[degree - back] + product[degree] * recurrence[back - 1]
            ) % MOD;
    product.resize(order);
    return product;
}

int64 nth_term(const vector<int64> &sequence, const vector<int64> &recurrence,
               unsigned long long index) {
    if (index < sequence.size()) return sequence[index];
    int order = recurrence.size();
    if (order == 0) return 0;
    vector<int64> result(order), base(order);
    result[0] = 1;
    if (order == 1) base[0] = recurrence[0];
    else base[1] = 1;
    while (index) {
        if (index & 1) result = combine(result, base, recurrence);
        base = combine(base, base, recurrence);
        index >>= 1;
    }
    int64 answer = 0;
    for (int i = 0; i < order; ++i)
        answer = (answer + result[i] * sequence[i]) % MOD;
    return answer;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, start, target;
    unsigned long long length;
    if (!(cin >> n >> m >> length >> start >> target)) return 0;
    --start; --target;
    struct Edge { int from, to; int64 weight; };
    vector<Edge> edges(m);
    for (auto &[from, to, weight] : edges) {
        cin >> from >> to >> weight;
        --from; --to;
    }
}

```

```

}

vector<int64> state(n), sequence(2 * n + 1);
state[start] = 1;
for (int step = 0; step <= 2 * n; ++step) {
    sequence[step] = state[target];
    vector<int64> next(n);
    for (auto [from, to, weight] : edges)
        next[to] = (next[to] + state[from] * weight) % MOD;
    state.swap(next);
}
vector<int64> recurrence = berlekamp_massey(sequence);
cout << nth_term(sequence, recurrence, length) << '\n';
}

```

6.5 Full Python solution

```

import sys

MOD = 998_244_353

def berlekamp_massey(sequence: list[int]) -> list[int]:
    current, saved = [1], [1]
    length, shift, saved_discrepancy = 0, 1, 1
    for position, term in enumerate(sequence):
        discrepancy = term
        for i in range(1, length + 1):
            discrepancy = (discrepancy + current[i] * sequence[position - i]) % MOD
        if discrepancy == 0:
            shift += 1
            continue
        previous = current[:]
        scale = discrepancy * pow(saved_discrepancy, MOD - 2, MOD) % MOD
        current += [0] * max(0, len(saved) + shift - len(current))
        for i, value in enumerate(saved):
            current[i + shift] = (current[i + shift] - scale * value) % MOD
        if 2 * length <= position:
            length = position + 1 - length
            saved = previous
            saved_discrepancy = discrepancy
            shift = 1
        else:
            shift += 1
    return [(-current[i]) % MOD for i in range(1, length + 1)]

def combine(left: list[int], right: list[int], recurrence: list[int]) -> list[int]:

```

```

order = len(recurrence)
product = [0] * (2 * order - 1)
for i, a in enumerate(left):
    for j, b in enumerate(right):
        product[i + j] = (product[i + j] + a * b) % MOD
for degree in range(2 * order - 2, order - 1, -1):
    value = product[degree]
    for back, coefficient in enumerate(recurrence, 1):
        product[degree - back] = (
            product[degree - back] + value * coefficient
        ) % MOD
return product[:order]

def nth_term(sequence: list[int], recurrence: list[int], index: int) -> int:
    if index < len(sequence):
        return sequence[index]
    order = len(recurrence)
    if order == 0:
        return 0
    result = [1] + [0] * (order - 1)
    base = [recurrence[0]] if order == 1 else [0, 1] + [0] * (order - 2)
    while index:
        if index & 1:
            result = combine(result, base, recurrence)
            base = combine(base, base, recurrence)
            index >>= 1
    return sum(result[i] * sequence[i] for i in range(order)) % MOD

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, walk_length, start, target = data[:5]
    start -= 1
    target -= 1
    edges = []
    index = 5
    for _ in range(m):
        u, v, weight = data[index : index + 3]
        index += 3
        edges.append((u - 1, v - 1, weight))
    state = [0] * n
    state[start] = 1
    sequence = []
    for _ in range(2 * n + 1):
        sequence.append(state[target])
        next_state = [0] * n
        for u, v, weight in edges:
            next_state[v] = (next_state[v] + state[u] * weight) % MOD

```

```
        state = next_state
    recurrence = berlekamp_massey(sequence)
    print(nth_term(sequence, recurrence, walk_length))

if __name__ == "__main__":
    main()
```

7 F. GCD-One Size Spectrum

7.1 First transform: exact gcd by Mobius

For divisor d , let $\text{cnt}[d]$ be the number of input values divisible by d . For a fixed subset size k , $C(\text{cnt}[d], k)$ counts subsets whose every value is divisible by d . Mobius inclusion-exclusion gives

```
answer[k] = sum_d mu[d] * C(cnt[d], k).
```

Computing this for every (d, k) is quadratic. We need to evaluate all k in one convolution.

7.2 Group equal counts and expose a convolution

Define

```
bucket[c] = sum(mu[d] for d with cnt[d]=c).
```

Then

```
answer[k]
= sum_{c>=k} bucket[c] * c! / (k! (c-k)!).
```

Factor out $1/k!$. Set

```
X[c] = bucket[c] * c!
Y[j] = 1/j!
```

The remaining sum is $\sum_{c>=k} X[c]Y[c-k]$. Reverse X by setting $RX[n-c]=X[c]$; convolution coefficient $n-k$ is exactly this sum because $(n-c)+(c-k)=n-k$. Multiply it by $1/k!$ to obtain the answer.

For 2,3,6, no singleton has gcd one, only pair {2,3} does, and the full triple does, producing 0 1 1.

7.3 Correctness proof

Lemma 1. The Mobius sum counts size- k subsets with gcd exactly one.

Proof. Fix a chosen subset with gcd g . Its total coefficient is $\sum_{d|g} \mu(d)$, which is one for $g=1$ and zero otherwise. QED.

Lemma 2. Convolution coefficient $n-k$, multiplied by inverse factorial k , equals the Mobius sum of Lemma 1.

Proof. The coefficient pairs reversed index $n-c$ only with index $c-k$. Its term is $\text{bucket}[c] * c! * 1/(c-k)!$; multiplying by $1/k!$ gives $\text{bucket}[c] C(c, k)$. Summing over valid c is precisely the grouped Mobius sum. QED.

Together the lemmas prove every printed spectrum entry.

7.4 Complexity and tests

Divisible counts and the Mobius sieve cost $O(M \log M)$ and $O(M)$ memory. One NTT costs $O(n \log n)$ with $O(n)$ memory. Test all ones, no gcd-one subset, duplicates, and brute all subsets for $n \leq 15$.

7.5 Full C++ solution

```
#include <bits/stdc++.h>
#include "../65_polynomial_algorithms_recurrences/ntt.hpp"
using namespace std;
using course_ntt::MOD;
using course_ntt::convolution;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<int> values(n);
    int maximum = 1;
    for (int &value : values) cin >> value, maximum = max(maximum, value);

    vector<int> frequency(maximum + 1);
    for (int value : values) ++frequency[value];
    vector<int> divisible(maximum + 1);
    for (int d = 1; d <= maximum; ++d)
        for (int multiple = d; multiple <= maximum; multiple += d)
            divisible[d] += frequency[multiple];

    vector<int> mu(maximum + 1), primes;
    vector<bool> composite(maximum + 1);
    mu[1] = 1;
```

```

for (int value = 2; value <= maximum; ++value) {
    if (!composite[value]) primes.push_back(value), mu[value] = -1;
    for (int prime : primes) {
        if (1LL * value * prime > maximum) break;
        composite[value * prime] = true;
        if (value % prime == 0) {
            mu[value * prime] = 0;
            break;
        }
        mu[value * prime] = -mu[value];
    }
}

vector<int> bucket(n + 1);
for (int d = 1; d <= maximum; ++d) {
    bucket[divisible[d]] += mu[d];
    if (bucket[divisible[d]] < 0) bucket[divisible[d]] += MOD;
    if (bucket[divisible[d]] >= MOD) bucket[divisible[d]] -= MOD;
}

vector<int> factorial(n + 1, 1), inverse_factorial(n + 1, 1);
for (int i = 1; i <= n; ++i)
    factorial[i] = static_cast<int>(1LL * factorial[i - 1] * i % MOD);
inverse_factorial[n] = course_ntt::power(factorial[n], MOD - 2);
for (int i = n; i >= 1; --i)
    inverse_factorial[i - 1] = static_cast<int>(1LL * inverse_factorial[i] * i % MOD);

vector<int> reversed(n + 1), inverse(n + 1);
for (int count = 0; count <= n; ++count) {
    reversed[n - count] = static_cast<int>(1LL * bucket[count] * factorial[count] % MOD);
    inverse[count] = inverse_factorial[count];
}
vector<int> product = convolution(reversed, inverse);
for (int size = 1; size <= n; ++size) {
    int answer = static_cast<int>(1LL * product[n - size] * inverse_factorial[size] % MOD);
    cout << answer << (size == n ? '\n' : ' ');
}
}

```

7.6 Full Python solution

```

import sys
from pathlib import Path

SECTION_65 = Path(__file__).resolve().parents[3] / "65_polynomial_algorithms_recurrences"
sys.path.insert(0, str(SECTION_65))
from ntt import MOD, convolution

```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    values = data[1 : 1 + n]
    maximum = max(values)
    frequency = [0] * (maximum + 1)
    for value in values:
        frequency[value] += 1
    divisible = [0] * (maximum + 1)
    for divisor in range(1, maximum + 1):
        divisible[divisor] = sum(frequency[divisor::divisor])

    mu = [0] * (maximum + 1)
    composite = bytearray(maximum + 1)
    primes: list[int] = []
    mu[1] = 1
    for value in range(2, maximum + 1):
        if not composite[value]:
            primes.append(value)
            mu[value] = -1
            for prime in primes:
                product = value * prime
                if product > maximum:
                    break
                composite[product] = 1
                if value % prime == 0:
                    mu[product] = 0
                    break
                mu[product] = -mu[value]

    bucket = [0] * (n + 1)
    for divisor in range(1, maximum + 1):
        bucket[divisible[divisor]] = (bucket[divisible[divisor]] + mu[divisor]) % MOD

    factorial = [1] * (n + 1)
    for i in range(1, n + 1):
        factorial[i] = factorial[i - 1] * i % MOD
    inverse_factorial = [1] * (n + 1)
    inverse_factorial[n] = pow(factorial[n], MOD - 2, MOD)
    for i in range(n, 0, -1):
        inverse_factorial[i - 1] = inverse_factorial[i] * i % MOD

    reversed_values = [0] * (n + 1)
    for count in range(n + 1):
        reversed_values[n - count] = bucket[count] * factorial[count] % MOD
    product = convolution(reversed_values, inverse_factorial)
    answers = [product[n - size] * inverse_factorial[size] % MOD for size in range(1, n + 1)]
    print(*answers)

```

```
if __name__ == "__main__":  
    main()
```

8 Stress-testing strategy

The randomized generators do not call the optimized references. They enumerate rotation orbits for A, gcds for B, rotations for C, bounded DP for D, direct walk transitions for E, and every subset for F. At this level, a reference algorithm and an oracle should fail for different reasons.

9 What to carry forward

The hardest two problems first compress a large structured state into a scalar sequence or a count histogram. Only after that compression does the advanced algebra become useful. This order—model, compress, then accelerate—is more reliable than searching the statement for the name of a technique.